# Introduction to Data Analysis Using Numpy

# NumPy

- "Numerical Python“ → NumPy
- NumPy is a Python library used for working with arrays.
- It also has functions for working in domain of linear algebra, Fourier transform, and matrices.
- NumPy was created in 2005 by Travis Oliphant. It is an open source project and you can use it freely
- Many other python libraries are built on NumPy

**Link:** http://www.numpy.org/
https://www.w3schools.com/python/numpy/default.asp

# NumPy

- In Python we have lists that serve the purpose of arrays, but they are slow to process.

- NumPy provides vectorization of mathematical operations on arrays and matrices which significantly improves the performance. It is up to 50x faster than traditional Python lists. Most of the  NumPy parts that require fast computation are written in C or C++

- Arrays are very frequently used in data science, where speed and resources are very important.

- NumPy introduces classes/ objects for multidimensional arrays and matrices, as well as functions that allow to easily perform advanced mathematical and statistical operations on those objects

**Link:** http://www.numpy.org/
https://www.w3schools.com/python/numpy/default.asp

# Install Numpy

*Install these libraries*
*py -m pip install numpy*
*py -m pip install pandas*
*py -m pip install matplotlib*

```
import numpy as np
```

NumPy is usually imported under the np alias.
**alias:** In Python alias are an alternate name for referring to the same thing.

# NumPy Creating Arrays

- **Create a NumPy ndarray Object**
- NumPy is used to work with arrays. The array object in NumPy is called ndarray.
- We can create a NumPy ndarray object by using the array() function.

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5])

print(arr)

print(type(arr))
```

```
[1 2 3 4 5]
<class 'numpy.ndarray'>
```

Use a list to create a NumPy array

```
import numpy as np

arr = np.array((1, 2, 3, 4, 5))

print(arr)
```

```
[1 2 3 4 5]
```

Use a tuple to create a NumPy array

# Dimensions in Arrays

**0-D Arrays**

0-D arrays, or Scalars, are the elements in an array. Each value in an array is a 0-D array.

```
import numpy as np

arr = np.array(42)

print(arr)
```

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5])

print(arr)
```

**1-D Arrays**

An array that has 0-D arrays as its elements is called uni-dimensional or 1-D array.
These are the most common and basic arrays.

**2-D Arrays**

An array that has 1-D arrays as its elements is called a 2-D array.
These are often used to represent matrix or 2nd order tensors.

```
import numpy as np
arr = np.array([[1, 2, 3], [4, 5, 6]])
print(arr)
```

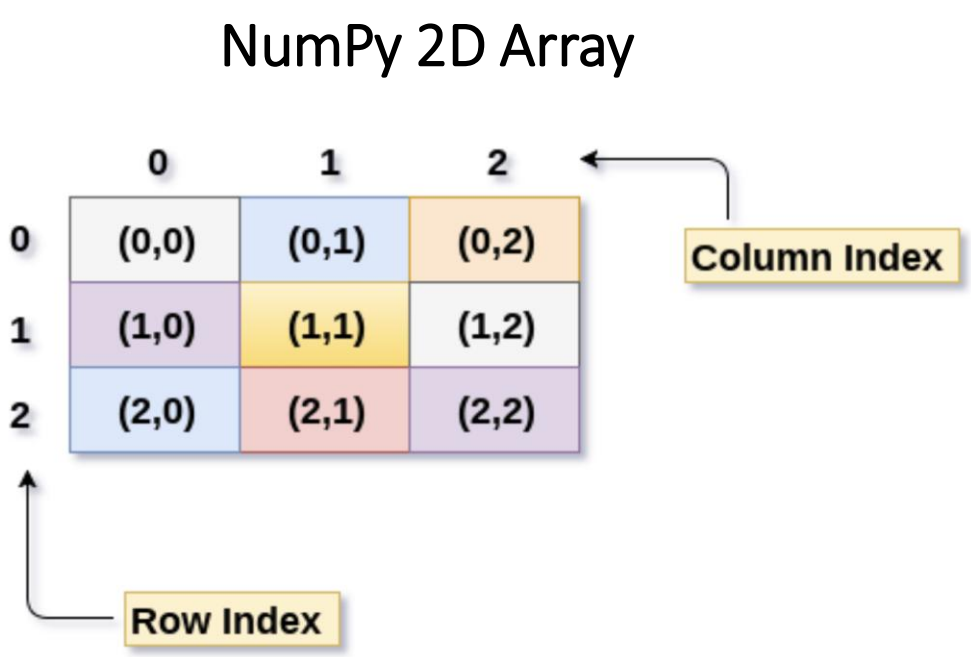
```
import numpy as np

arr = np.array([[[1, 2, 3], [4, 5, 6]], [[1, 2, 3], [4, 5, 6]]])

print(arr)
```

**3-D arrays**

An array that has 2-D arrays (matrices) as its elements is called 3-D array.
These are often used to represent a 3rd order tensor.

# NumPy Array Indexing

- Similar to Python lists, you can access an array element by referring to its index number.
- The indexes in NumPy arrays start with 0, meaning that the first element has index 0, and the second has index 1 etc.
- Use negative indexing to access an array from the end.




```
arr = np.array([[1,2,3],
                [4,5,6],
                [7,8,9]])
print(arr)
print(arr[1,1])
print(arr[1,:]) #This will return all elements of 1st row in the form of array
print(arr[1:3,1:3]) # return multiple rows and columns
print(arr[1][2])
```

```
[[1 2 3]
 [4 5 6]
 [7 8 9]]
5
[4 5 6]
[[5 6]
 [8 9]]
6
```

# NumPy Array Indexing

**Access 2-D Arrays**

- To access elements from 2-D arrays we can use comma separated integers representing the dimension and the index of the element.
- Think of 2-D arrays like a table with rows and columns, where the row represents the dimension and the index represents the column.

```
import numpy as np

arr = np.array([[1,2,3,4,5], [6,7,8,9,10]])

print('2nd element on 1st row: ', arr[0, 1])
```

First Dimension : 0 → [1,2,3,4,5]
Second dimension: 1 → 2

---

```
import numpy as np

arr = np.array([[[1, 2, 3], [4, 5, 6]], [[7, 8, 9], [10, 11, 12]]])

print(arr[0, 1, 2])
```

First Dimension : 0 → [[1, 2, 3], [4, 5, 6]]
Second dimension: 1 → [4, 5, 6]
Third dimension: 2 → 6

**Access 3-D Arrays**

To access elements from 3-D arrays we can use comma separated integers representing the dimensions and the index of the element.

# NumPy Array Slicing

- We pass slice instead of index like this: [*start*:*end*]. End is not included.
- We can also define the step, like this: [*start*:*end*:*step*]. Use the step value to determine the step of the slicing
- If we don't pass start, it is considered 0
- If we don't pass end, it is considered length of array in that dimension
- If we don't pass step, it is considered 1
- Use the minus operator to refer to an index from the end

```
import numpy as np

arr = np.array([[1, 2, 3, 4, 5], [6, 7, 8, 9, 10]])

print(arr[0:2, 1:4])
```

```
[[2 3 4]
 [7 8 9]]
```

**Slicing 2-D Arrays**

```
import numpy as np

arr = np.array([[1, 2, 3, 4, 5], [6, 7, 8, 9, 10]])

print(arr[0:2, -3])
```

```
[3 8]
```

# Numpy array vs. list

- Similarities
  - Access elements by index. a[0]
  - Access a range of elements. a[0:3] (note upper bound is not included)
  - Use loops. for x in a:

```
a = np.array([1,2,3,4,5])
print("numpy array, a[0] = {}".format(a[0]))
print("numpy array, a[1:3] = {}".format(a[1:3]))
for x in a:
    print("numpy array element: ", x)
```

```
numpy array, a[0] = 1
numpy array, a[1:3] = [2 3]
numpy array element:  1
numpy array element:  2
numpy array element:  3
numpy array element:  4
numpy array element:  5
```

```
l = [1,2,3,4,5]
print("list, l[0] = {}".format(l[0]))
print("list, l[1:3] = {}".format(l[1:3]))
for x in l:
    print("list element: ", x)
```

```
list, l[0] = 1
list, l[1:3] = [2, 3]
list element:  1
list element:  2
list element:  3
list element:  4
list element:  5
```

# Numpy array vs. list

- Differences
  - For a Numpy array, each element should have the same data type (string, int, boolean, etc). For a Python list, you can put different data types together.
  - Numpy has convenient functions. mean(), std(). List also has similar functions, but Numpy is faster.
  - Numpy can be multi-dimensional, similar to list of list in python.

```
a = np.array([[1,2,3], [4,5,6]])
print("mean value over all elements: ", a.mean())
print("mean value for each row: ", a.mean(axis=1))
print("mean value for each column: ", a.mean(axis=0))
```

```
mean value over all elements:  3.5
mean value for each row:  [2. 5.]
mean value for each column:  [2.5 3.5 4.5]
```

```
l = [[1,2,3], [4,5,6]]
print("mean value over all elements: ", (sum(l[0])+sum(l[1]))/6)
```

```
mean value over all elements:  3.5
```

# Vectorized Operations

- A Numpy vector is a list of elements.
  - Vector addition.
  - Multiplying by a scalar
  - More vectorized operations.
    - Math operation: +, -, *, /, **,
    - Logical operations: And:&, Or:|, Not:~. Only valid for boolean. If use it on integers, then it's bitwise operation .
    - Comparison operations: >, >=, <=, <, ==, !=.

```
a = np.array([1,2,3])
b = np.array([4,5,6])
print("a+b = {}".format(a+b))
print("a*b = {}".format(a*b))
print("a*3 = {}".format(a*3))
print("a&b = {}".format(a&b))
print("a>b = {}".format(a>b))
```

```
a+b = [5 7 9]
a*b = [ 4 10 18]
a*3 = [3 6 9]
a&b = [0 0 2]
a>b = [False False False]
```

**Numpy Binary operations**
https://numpy.org/doc/stable/reference/routines.bitwise.html

**Bitwise operations between two integers**

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 7 | = | 0 | 1 | 1 | 1 | | |
| 4 | = | 0 | 1 | 0 | 0 | | |
| 7 AND 4 | = | 0 | 1 | 0 | 0 | = | 4 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 7 | = | 0 | 1 | 1 | 1 | | |
| 4 | = | 0 | 1 | 0 | 0 | | |
| 7 OR 4 | = | 0 | 1 | 1 | 1 | = | 7 |

# NumPy index arrays

- An array of booleans can be an index array

```
# using index arrays
a = np.array([1, 2, 3, 4])
b = np.array([True, True, False, False])
print(a[b])
print(a[a>2])
# creating the index array using vectorized operations
b = a >2
print(b)
# array of booleans can be used with numpy functions
print(b.mean())
```

```
[1 2]
[3 4]
[False False  True  True]
0.5
```

# + vs. +=

- += operates in-place while + does not.
- That is, += stores all the values in the same place as the original values rather than creating a new array.

```
# case 1: both a and b refer to the same array
a = np.array([1,2,3])
b = a
a += 1
print(b)
```

```
[2 3 4]
```

```
# case 2: both a and b refer to the same array,
# but later a is reassigned, which creates a new array.
# Then a refers to a new array while b refers
# to the original array
a = np.array([1,2,3])
b = a
a = a + 1
print(b)
```

```
[1 2 3]
```

# In-Place vs. Not In-Place

- When taking a slice, numpy doesn't create a new array.
- Instead it refers to a what's called a view of the original array.
- If the slice is modified, then the original array is modified as well. So slice in numpy is very fast.
- Note this is different from python list.

```
# try numpy array
a = np.array([1,2,3,4])
# take a slice, numpy doesn't create a new array.
Slice = a[:3]
Slice[0] = 10
print(a)
```

```
[10  2  3  4]
```

```
a = [1,2,3,4]
Slice = a[:3]
Slice[0] = 10
print(a)
```

```
[1, 2, 3, 4]
```

# Advanced indexing and index tricks

- NumPy offers more indexing facilities than regular Python sequences.
- In addition to indexing by integers and slices, arrays can be indexed by arrays of integers and arrays of booleans.

a=[ 0 1 4 9 16 25 36 49 64 81 100 121]

```
a = np.arange(12)**2  # the first 12 square numbers
i = np.array([1, 1, 3, 8, 5])  # an array of indices
print(a[i])  # the elements of `a` at the positions `i`
j = np.array([[3, 4], [9, 7]])  # a bidimensional array of indices
print(a[j])  # the same shape as `j`
```

```
[ 1  1  9 64 25]
[[ 9 16]
 [81 49]]
```

```
>>> np.arange(3)
array([0, 1, 2])
>>> np.arange(3.0)
array([ 0.,  1.,  2.])
>>> np.arange(3,7)
array([3, 4, 5, 6])
>>> np.arange(3,7,2)
array([3, 5])
```

Similar to the Python range function, numpy.arange([*start*, ]*stop*, [*step*, ]*dtype=None*, *, *like=None*) Return evenly spaced values within a given interval.

# Sorting elements

- Sorting an element is simple with np.sort() with specific axis.

```
>>> a = np.array([[1,4],[3,1]])
>>> np.sort(a)                # sort along the last axis
array([[1, 4],
       [1, 3]])
>>> np.sort(a, axis=None)     # sort the flattened array
array([1, 1, 3, 4])
>>> np.sort(a, axis=0)        # sort along the first axis
array([[1, 1],
       [3, 4]])
```

# Shape of the array

- ndarray.ndim: display the number of axes, or dimensions of the array.
- ndarray.size: display the total number of elements of the array
- ndarray.shape: display a tuple of integers that indicate the number of elements stored along each dimension of the array.

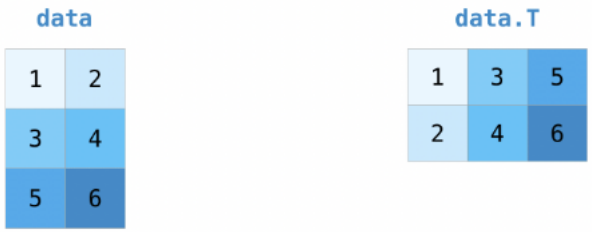
```
array_example = np.array([[[0, 1, 2, 3],
                           [4, 5, 6, 7]],

                          [[0, 1, 2, 3],
                           [4, 5, 6, 7]],

                          [[0 ,1 ,2, 3],
                           [4, 5, 6, 7]]])
print(array_example.ndim)
print(array_example.size)
print(array_example.shape)
```

```
3
24
(3, 2, 4)
```

# Transposing and reshaping a matrix

- Transpose: Reverse or permute the axes of an array; returns the modified array.

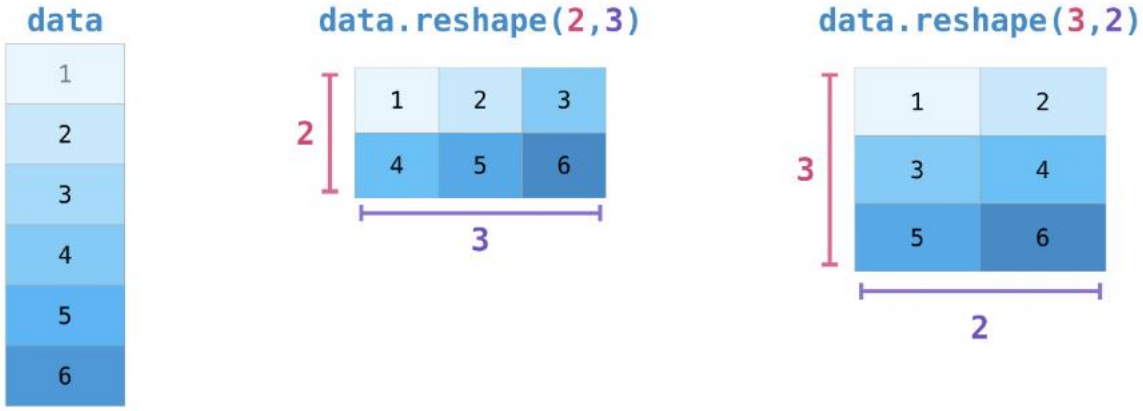
  - Transposing a 1-D array returns an unchanged view of the original array.
  - For an array a with two axes, transpose(a) gives the matrix transpose.




```
arr = np.array([[1,2,3],[4,5,6]])
print(arr)
print(arr.T)
```

```
[[1 2 3]
 [4 5 6]]
[[1 4]
 [2 5]
 [3 6]]
```

- Reshape: Gives a new shape to an array without changing its data




```
arr = np.array([1,2,3,4,5,6])
print(arr.reshape(2, 3))
print(arr.reshape(3, 2))
```

```
[[1 2 3]
 [4 5 6]]
[[1 2]
 [3 4]
 [5 6]]
```